Roll no : 241501199
Name : Sherwin Richie J

CS23532-COMPUTER NETWORKS-LAB MANUAL

PRACTICAL - 6

AIM:

To write and implement a program for error detection and correction at the Data Link Layer using the Hamming Code technique, and to perform a test run with an input data stream to verify the error detection and correction capability.

PROCEDURE:

1. Create an input file containing the data stream that needs to be transmitted.
2. Develop the sender program to read each character from the input data stream.
3. Convert the input data into binary form and add the required Hamming parity bits at appropriate positions.
4. Generate the Hamming code and transmit the encoded data through the communication channel.
5. At the receiver side, read the received Hamming-coded data and perform parity checking.
6. Calculate the error position (syndrome) to detect whether an error has occurred during transmission.
7. If a single-bit error is detected, correct the erroneous bit and decode the Hamming code to obtain the original data stream.

PROGRAM CODE:

Sender.c

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>
```

Roll no : 241501199
Name : Sherwin Richie J

```
int calculateParityBits(int dataBits)
{
    int r = 0;

    while ((1 << r) < (dataBits + r + 1))
        r++;

    return r;
}

void encodeHamming(unsigned char data, FILE *channel)
{
    int hamming[13] = {0};
    int dataIndex = 0;

    for (int pos = 1; pos <= 12; pos++)
    {
        if (pos == 1 || pos == 2 || pos == 4 || pos == 8)
            continue;

        hamming[pos] = (data >> (7 - dataIndex)) & 1;
        dataIndex++;
    }
```

```
    hamming[1] = hamming[3] ^ hamming[5] ^ hamming[7] ^
                 hamming[9] ^ hamming[11];

    hamming[2] = hamming[3] ^ hamming[6] ^ hamming[7] ^
                 hamming[10] ^ hamming[11];

    hamming[4] = hamming[5] ^ hamming[6] ^ hamming[7] ^
                 hamming[12];

    hamming[8] = hamming[9] ^ hamming[10] ^ hamming[11] ^
                 hamming[12];

    for (int i = 1; i <= 12; i++)
        fprintf(channel, "%d", hamming[i]);
}

int main()
{
    FILE *input;
    FILE *channel;
    char ch;

    input = fopen("input.txt", "r");
```

```
if (input == NULL)
{
   printf("Error: Could not open input.txt\n");
   return 1;
}

channel = fopen("channel.txt", "w");

if (channel == NULL)
{
   printf("Error: Could not create channel.txt\n");
   fclose(input);
   return 1;
}

printf("========== HAMMING CODE SENDER ==========\n\n");

printf("Input text:\n");

while ((ch = fgetc(input)) != EOF)
   putchar(ch);

rewind(input);
```

```
    while ((ch = fgetc(input)) != EOF)
        encodeHamming((unsigned char)ch, channel);

    fclose(input);
    fclose(channel);

    printf("\n\nHamming encoded data has been written to channel.txt\n");

    printf("\nChannel data:\n");

    channel = fopen("channel.txt", "r");

    while ((ch = fgetc(channel)) != EOF)
        putchar(ch);

    fclose(channel);

    printf("\n\nSender completed successfully.\n");

    return 0;
}
```

Receiver.c

```
#include <stdio.h>
```

Roll no : 241501199
Name : Sherwin Richie J

```
#include <stdlib.h>

#include <string.h>


int main()

{

   FILE *channel;

   char bits[100000];

   int length = 0;


   channel = fopen("channel.txt", "r");


   if (channel == NULL)

   {

      printf("Error: Could not open channel.txt\n");

      return 1;

   }


   char ch;


   while ((ch = fgetc(channel)) != EOF)

   {

      if (ch == '0' || ch == '1')

      {

         bits[length++] = ch;
```

```
    }
}

bits[length] = '\0';

fclose(channel);

printf("========== HAMMING CODE RECEIVER ==========\n\n");

printf("Received channel data:\n%s\n\n", bits);

if (length % 12 != 0)
{
    printf("Error: Invalid Hamming data.\n");
    return 1;
}

printf("Checking for errors...\n\n");

printf("Corrected/Decoded output:\n");

for (int block = 0; block < length; block += 12)
{
    int hamming[13];
```

Roll no : 241501199
Name : Sherwin Richie J

```
for (int i = 1; i <= 12; i++)
{
   hamming[i] = bits[block + i - 1] - '0';
}

int p1 = hamming[1] ^ hamming[3] ^ hamming[5] ^
       hamming[7] ^ hamming[9] ^ hamming[11];

int p2 = hamming[2] ^ hamming[3] ^ hamming[6] ^
       hamming[7] ^ hamming[10] ^ hamming[11];

int p4 = hamming[4] ^ hamming[5] ^ hamming[6] ^
       hamming[7] ^ hamming[12];

int p8 = hamming[8] ^ hamming[9] ^ hamming[10] ^
       hamming[11] ^ hamming[12];

int errorPosition = p1 * 1 +
                p2 * 2 +
                p4 * 4 +
                p8 * 8;

if (errorPosition == 0)
```

```
{
   printf("[No error in block %d] ", (block / 12) + 1);
}
else if (errorPosition <= 12)
{
   printf("[Error in block %d at position %d - Corrected] ",
         (block / 12) + 1, errorPosition);

   hamming[errorPosition] =
      hamming[errorPosition] ^ 1;
}
else
{
   printf("[Invalid error detected] ");
}

int data = 0;

int dataPositions[] = {
   3, 5, 6, 7,
   9, 10, 11, 12
};

for (int i = 0; i < 8; i++)
```

```
        {
            data = (data << 1) | hamming[dataPositions[i]];
        }

        printf("%c", (char)data);
    }

    printf("\n\nReceiver completed successfully.\n");

    return 0;
}
```

Compilation and Execution

```
gcc sender.c -o sender
./sender
gcc receiver.c -o receiver
./receiver
```

Output

Sender Output

```
========== HAMMING CODE SENDER ==========

Input text:
HELLO WORLD
```

Roll no : 241501199
Name : Sherwin Richie J

Hamming encoded data has been written to channel.txt

Sender completed successfully.

Receiver Output

========== HAMMING CODE RECEIVER ==========

Checking for errors...

Corrected/Decoded output:

[No error in block 1] H

[No error in block 2] E

[No error in block 3] L

[No error in block 4] L

[No error in block 5] O

[No error in block 6]

[No error in block 7] W

[No error in block 8] O

[No error in block 9] R

[No error in block 10] L

[No error in block 11] D

Receiver completed successfully.

Roll no : 241501199
Name : Sherwin Richie J

Result

Thus, the Hamming Code sender and receiver programs were successfully implemented to encode the input data, detect and correct transmission errors, and recover the original message at the receiver end.

Roll no : 241501199
Name : Sherwin Richie J